

Министерство образования и науки Российской Федерации
Санкт-Петербургский государственный политехнический университет

—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1**

«Организация данных и разграничение доступа»

по дисциплине «Системы управления базами данных»

Выполнил
студент гр. 5131001/20502

<подпись>

Улановский Г. А.

студент гр. 5151001/20202

<подпись>

Игнатьев И.В.

Проверил
ассистент

<подпись>

Пономарев Д. А.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1	Формулировка задания	3
2	Теоретические сведения.....	4
2.1	Основы организации данных в СУБД.....	4
2.2	Разграничение доступа в СУБД.....	4
2.3	Роли, привилегии и политики безопасности строк	6
3	Результаты работы	8
3.1	Разработанная схема данных	8
3.2	Разработанная матрица доступа	9
3.3	Код реализации на SQL	11
3.4	Проверка разграничения доступа.....	15
3.5	Итоговая матрица доступа	20
4	Ответы на контрольные вопросы	22
4.1	Какой тип разграничения доступа, с точки зрения грануляции, реализован в СУБД, на которой выполнялась работа?	22
4.2	Какой тип гранулированного доступа реализует предложение GRANT? ..	22
4.3	Какой тип гранулированного доступа реализует предложение POLICY? ..	22
4.4	Что происходит при совмещении нескольких политик доступа в отношении одной ячейки (записи) для одного и того же пользователя?	23
4.5	Кто обладает правом разграничения прав доступа в рассматриваемой СУБД? Какие права для этого необходимы?	23
4.6	Есть ли какие-либо ограничения применения политик?	23
4.7	К каким объектам может быть применено предложение GRANT?.....	24
4.8	К каким объектам может быть применено предложение POLICY?	24
4.9	Какие пользователи и роли изначально определены в рассматриваемом сервере СУБД? У каких из них заданы пароли по умолчанию?	24
4.10	Как разграничить доступ пользователей к функциям? Какие права при этом должны быть также предоставлены пользователю и должны ли?	25
4.11	Можно ли дать пользователям доступ на чтение к отдельным таблицам системного каталога сервера СУБД?	25
5	Выводы.....	27
	Список используемых источников.....	28

1 ФОРМУЛИРОВКА ЗАДАНИЯ

Цель лабораторной работы — получение навыков создания схемы данных и базового разграничения доступа при работе с СУБД.

В ходе лабораторной работы необходимо реализовать заданные отношения, определить права доступа пользователей к данным и настроить разграничение доступа с использованием встроенных средств PostgreSQL: ролей, привилегий GRANT/REVOKE и политик безопасности строк Row Level Security.

Для выполнения лабораторной работы был получен вариант № 8. Вариант предусматривает разработку базы данных для хранения документов организации и журнала операций с этими документами, данные которого представлены в таблице 1.

Таблица 1 – Вариант задания №8

Описание данных	Описание прав доступа
Отношение 1 Тип документа (АК), Номер документа (РК), ФИО ответственного (АК), Содержание документа (JSON), Внутреннее подразделение (АК), Дата ввода в действие (АК), Дата окончания действия если есть, Степень секретности если есть.	Сотрудник видит все несекретные документы (отношение 1), но не операции с ними (отношение 2). Сотрудник видит все секретные документы, в которых он назначен ответственным. Сотрудник может изменять срок действия документов, в которых он назначен ответственным.
Отношение 2 ФИО сотрудника (РК), Номер документа (РК, FK), Дата изменения (РК), Операция с документом.	Сотрудник видит все операции с документами, которые выполнил он или, если они выполнены в отношении документов, за которые он ответственен. Сотрудник может изменять поле «операция с документом» в отношении 2 для своих записей.

В качестве тестовых данных в работе использовались обезличенные идентификаторы сотрудников: Сотрудник_А, Сотрудник_Б, Сотрудник_В, Сотрудник_Г и Сотрудник_Д. Такой подход позволяет проверить работу разграничения доступа без использования персональных данных.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Основы организации данных в СУБД

Система управления базами данных предназначена для хранения, обработки и защиты структурированных данных. В реляционных СУБД данные представляются в виде отношений, то есть таблиц. Каждая таблица состоит из строк и столбцов: строки соответствуют отдельным записям, а столбцы — атрибутам этих записей.

При проектировании базы данных обычно выделяют несколько уровней представления данных. На концептуальном уровне описывается предметная область и основные сущности. На логическом уровне эти сущности преобразуются в таблицы, определяются атрибуты, ключи и связи между отношениями. На физическом уровне задаются особенности хранения и обработки данных средствами конкретной СУБД.

Для однозначной идентификации записей в таблицах используются ключи. Первичный ключ служит основным идентификатором строки и не должен принимать повторяющиеся значения. Альтернативный ключ также задает уникальность записей, но не используется как основной идентификатор. Внешний ключ применяется для описания связи между таблицами и обеспечивает ссылочную целостность данных.

Поддержание целостности данных является одной из основных задач СУБД. Для этого используются ограничения первичного ключа, уникальности, внешнего ключа, проверки значений и обязательности заполнения полей. Такие ограничения позволяют предотвратить появление некорректных или противоречивых записей в базе данных.

2.2 Разграничение доступа в СУБД

Разграничение доступа применяется для ограничения действий пользователей в соответствии с их полномочиями. В информационных системах разные пользователи могут иметь разные права: один пользователь может только

просматривать данные, другой — изменять отдельные поля, третий — выполнять административные действия.

Обычно права доступа к данным рассматриваются через набор CRUD-операций:

Create — создание новых записей;

Read — чтение данных;

Update — изменение существующих записей;

Delete — удаление записей.

В языке SQL этим операциям соответствуют команды INSERT, SELECT, UPDATE и DELETE. Однако на практике разграничение доступа часто требуется задавать не только на уровне таблицы целиком. Пользователь может иметь право читать только отдельные атрибуты таблицы или видеть только часть строк, удовлетворяющих определенному условию.

По степени детализации можно выделить несколько уровней разграничения доступа:

- уровень объектов, когда права задаются на таблицу, представление, схему или другой объект базы данных;

- уровень столбцов, когда доступ разрешается только к отдельным атрибутам таблицы;

- уровень строк, когда пользователь видит или изменяет только те записи, которые соответствуют заданному правилу;

- уровень ячеек, когда доступ определяется одновременно строкой и столбцом.

Разграничение доступа на уровне объектов и столбцов обычно реализуется стандартными средствами SQL. Разграничение на уровне строк требует дополнительных механизмов, так как оно зависит не только от структуры таблицы, но и от содержимого конкретных записей.

2.3 Роли, привилегии и политики безопасности строк

В PostgreSQL управление доступом строится на ролях и привилегиях. Роль является субъектом безопасности, которому могут быть назначены права. Роль может использоваться как учетная запись пользователя с возможностью входа в систему или как групповая роль, объединяющая набор привилегий.

Для выдачи прав используется оператор GRANT. Он позволяет разрешить выполнение определенных действий над объектами базы данных. Например, пользователю или роли может быть выдано право на чтение таблицы, изменение данных или выполнение функции. Для отзыва ранее выданных прав используется оператор REVOKE.

PostgreSQL поддерживает выдачу прав не только на таблицу целиком, но и на отдельные столбцы. Это позволяет ограничить пользователя так, чтобы он мог читать или изменять только определенные атрибуты. Такой механизм полезен, если часть данных должна быть скрыта от пользователя или если пользователю разрешено изменять только одно поле из всей записи.

Для ограничения доступа на уровне строк в PostgreSQL используется механизм Row Level Security. После включения RLS для таблицы СУБД начинает применять политики безопасности строк. Политика содержит логическое условие, которое проверяется для каждой строки таблицы.

При чтении данных используется выражение USING. Оно определяет, какие строки будут доступны пользователю при выполнении SELECT. Для операций изменения может использоваться выражение WITH CHECK, которое проверяет, допустимо ли новое состояние строки после выполнения INSERT или UPDATE.

Стандартные средства разграничения доступа имеют ограничения. Оператор GRANT задает права на уровне объектов и столбцов, а политики RLS — на уровне строк. Поэтому точное разграничение доступа на уровне отдельных ячеек может требовать сочетания нескольких механизмов или использования дополнительных средств, например представлений, функций или триггеров.

Также необходимо учитывать, что при совмещении нескольких ролей итоговый набор прав пользователя может расширяться. Если пользователь наследует права от нескольких ролей, он может получить доступ, который не был очевиден при рассмотрении каждой роли отдельно. Поэтому при проектировании системы разграничения доступа важно анализировать не только отдельные права, но и их возможные комбинации.

Таким образом, базовое разграничение доступа в PostgreSQL может строиться на сочетании ролей, привилегий GRANT/REVOKE и политик безопасности строк. Роли определяют субъектов доступа, привилегии задают разрешенные операции над объектами и столбцами, а политики RLS ограничивают доступ к отдельным строкам таблицы.

3 РЕЗУЛЬТАТЫ РАБОТЫ

В ходе выполнения лабораторной работы была разработана схема базы данных для варианта № 8. База данных предназначена для хранения сведений о документах организации и журнале операций, выполняемых с этими документами.

Работа выполнялась в среде WSL Ubuntu. СУБД PostgreSQL 16.13 была запущена в Docker-контейнере. Для выполнения SQL-скриптов использовался клиент psql, а для визуальной проверки структуры базы данных и просмотра таблиц применялся DBeaver.

В качестве тестовых данных использовались обезличенные идентификаторы сотрудников: Сотрудник_А, Сотрудник_Б, Сотрудник_В, Сотрудник_Г и Сотрудник_Д. Такой набор данных позволяет проверить разграничение доступа без использования персональных данных.

3.1 Разработанная схема данных

Для выполнения варианта № 8 были разработаны два отношения: documents и document_operations.

Таблица documents хранит сведения о документах организации. В качестве первичного ключа используется номер документа. Для дополнительного контроля уникальности задан альтернативный ключ, включающий тип документа, ответственного сотрудника, внутреннее подразделение и дату ввода документа в действие.

Таблица document_operations хранит журнал операций с документами. Первичный ключ таблицы является составным и включает сотрудника, номер документа и дату изменения. Поле document_number является внешним ключом и ссылается на таблицу documents.

Таблица 2 – Разработанные отношения базы данных

Отношение	Назначение	Основные атрибуты	Ключи и ограничения
documents	Хранение сведений о документах организации	document_number, document_type, responsible_employee, document_content,	document_number — PK; document_type, responsible_employee, internal_department,

		internal_department, effective_from, effective_until, secrecy_level	effective_from — АК; проверка корректности дат; проверка допустимых значений secrecy_level
document_operations	Хранение операций, выполненных с документами	employee_name, document_number, change_date, operation_description	employee_name, document_number, change_date — составной ПК; document_number — FK на documents(document_number)

Реляционная схема базы данных представлена на рисунке 1.

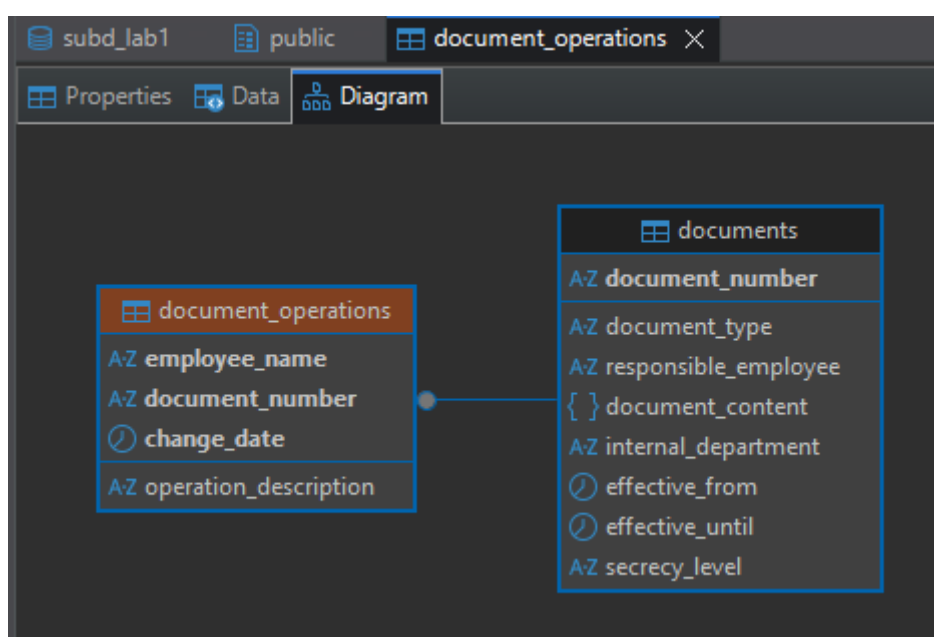


Рисунок 1 – Реляционная схема базы данных

На схеме видно, что таблица document_operations связана с таблицей documents по полю document_number. Такая связь обеспечивает невозможность добавления операции для документа, отсутствующего в таблице документов.

3.2 Разработанная матрица доступа

На основании условий варианта была разработана матрица доступа. Права доступа определялись из набора CRUD-операций. Для данной работы использовались следующие обозначения: R — чтение данных, U — изменение данных, «-» — отсутствие доступа.

Так как вариант предусматривает одного типа пользователя — сотрудника, ограничения доступа задаются не только по атрибутам, но и по строкам. Видимость строк зависит от того, является ли документ секретным, является ли текущий пользователь ответственным за документ, а также выполнял ли он конкретную операцию с документом.

Таблица 3 – Исходная матрица доступа

Отношение	Атрибут или условие	Сотрудник
documents	document number	R
documents	document type	R
documents	responsible employee	R
documents	document content	R
documents	internal department	R
documents	effective from	R
documents	effective until	R/U
documents	secrecy level	R
document_operations	employee name	R
document_operations	document number	R
document_operations	change date	R
document_operations	operation description	R/U
documents	Видимые строки	Все несекретные документы; секретные документы только при <code>responsible_employee = current_user</code>
document_operations	Видимые строки	Свои операции; операции по документам, за которые пользователь ответственен
documents	Ограничение изменения	<code>effective_until</code> можно изменять только для документов, где <code>responsible_employee = current_user</code>
document_operations	Ограничение изменения	<code>operation_description</code> можно изменять только для записей, где <code>employee_name = current_user</code>

Из матрицы следует, что основное разграничение доступа должно выполняться не только на уровне столбцов, но и на уровне строк. Поэтому для реализации были использованы столбцовые привилегии PostgreSQL и политики безопасности строк RLS.

3.3 Код реализации на SQL

Код реализации был разделен на несколько SQL-скриптов. Такое разделение позволяет отдельно показать этапы создания структуры базы данных, заполнения тестовыми данными, создания пользователей и настройки разграничения доступа.

На первом этапе были созданы таблицы `documents` и `document_operations`. Таблица `documents` предназначена для хранения сведений о документах, а таблица `document_operations` — для хранения журнала операций с документами. Фрагмент SQL-кода создания таблиц приведен в листинге 1.

Листинг 1 – Создание таблиц базы данных

```
CREATE TABLE documents
(
    document_number      text PRIMARY KEY,
    document_type        text NOT NULL,
    responsible_employee text NOT NULL,
    document_content     jsonb NOT NULL,
    internal_department  text NOT NULL,
    effective_from       date NOT NULL,
    effective_until      date,
    secrecy_level        text,

    CONSTRAINT uq_documents_alternative_key UNIQUE
    (
        document_type,
        responsible_employee,
        internal_department,
        effective_from
    ),
    CONSTRAINT chk_documents_dates CHECK
    (
        effective_until IS NULL OR effective_until >= effective_from
    ),
    CONSTRAINT chk_documents_secrecy CHECK
    (
        secrecy_level IS NULL OR secrecy_level IN ('ДСП', 'секретно')
    )
);
CREATE TABLE document_operations
(
    employee_name      text NOT NULL,
    document_number    text NOT NULL REFERENCES documents(document_number)
                        ON UPDATE CASCADE
                        ON DELETE CASCADE,
    change_date        timestamp NOT NULL,
    operation_description text NOT NULL,
    CONSTRAINT pk_document_operations PRIMARY KEY
    (
        employee_name,
        document_number,
        change_date
    )
)
```

```
);
```

В таблице documents задан первичный ключ document_number, альтернативный ключ по типу документа, ответственному сотруднику, подразделению и дате ввода в действие, а также ограничения для проверки дат и допустимых значений степени секретности. В таблице document_operations задан составной первичный ключ и внешний ключ на таблицу documents.

На следующем этапе таблицы были заполнены обезличенными тестовыми данными. Для проверки разграничения доступа были добавлены как несекретные документы, так и документы со степенью секретности.

Листинг 2 – Тестовые данные

```
INSERT INTO documents
(
    document_number,
    document_type,
    responsible_employee,
    document_content,
    internal_department,
    effective_from,
    effective_until,
    secrecy_level
)
VALUES
(
    'DOC-001',
    'Регламент',
    'Сотрудник_А',
    '{"title": "Регламент эксплуатации рабочих станций", "version": 1,
"summary": "Общие правила эксплуатации"}',
    'Подразделение_1',
    '2024-01-10',
    '2026-01-10',
    NULL
),
(
    'DOC-002',
    'План реагирования',
    'Сотрудник_А',
    '{"title": "План реагирования на инциденты", "version": 2, "summary":
"Порядок действий при инцидентах"}',
    'Подразделение_1',
    '2024-02-01',
    '2026-02-01',
    'секретно'
),
(
    'DOC-006',
    'Служебная записка',
    'Сотрудник_В',
    '{"title": "Служебная записка по внутренней проверке", "version": 1,
"summary": "Материалы внутренней проверки"}',
    'Подразделение_3',
    '2024-06-01',
```

```
'2025-06-01',  
'секретно'  
);
```

В листинге приведена часть тестовых данных. Полный набор данных включал восемь документов. Значение NULL в поле `secrecy_level` использовалось для несекретных документов, а значения «ДСП» и «секретно» — для документов с ограничением доступа.

Для разграничения доступа была создана групповая роль `document_employee` и пять пользователей, соответствующих обезличенным сотрудникам. Все пользователи были включены в общую роль.

Листинг 3 – Создание роли и пользователей

```
CREATE ROLE document_employee;  
  
CREATE ROLE "Сотрудник_А" LOGIN PASSWORD 'lab1pass';  
CREATE ROLE "Сотрудник_Б" LOGIN PASSWORD 'lab1pass';  
CREATE ROLE "Сотрудник_В" LOGIN PASSWORD 'lab1pass';  
CREATE ROLE "Сотрудник_Г" LOGIN PASSWORD 'lab1pass';  
CREATE ROLE "Сотрудник_Д" LOGIN PASSWORD 'lab1pass';  
  
GRANT document_employee TO  
    "Сотрудник_А",  
    "Сотрудник_Б",  
    "Сотрудник_В",  
    "Сотрудник_Г",  
    "Сотрудник_Д";  
  
GRANT USAGE ON SCHEMA public TO document_employee;
```

Использование общей роли позволяет назначать базовые права доступа централизованно. Индивидуальная логика доступа определяется не отдельными наборами привилегий, а политиками безопасности строк, использующими имя текущего пользователя.

Права доступа были выданы с учетом разработанной матрицы. Пользователям разрешено читать таблицы документов и операций, но изменение разрешено только для отдельных столбцов.

Листинг 4 – Выдача прав доступа

```
REVOKE ALL ON documents FROM PUBLIC;  
REVOKE ALL ON document_operations FROM PUBLIC;  
  
GRANT SELECT ON documents TO document_employee;  
GRANT UPDATE (effective_until) ON documents TO document_employee;  
  
GRANT SELECT ON document_operations TO document_employee;  
GRANT UPDATE (operation_description) ON document_operations TO  
document_employee;
```

Право UPDATE было выдано не на таблицы целиком, а только на отдельные столбцы: effective_until для таблицы documents и operation_description для таблицы document_operations. За счет этого пользователи не могут изменять номер документа, степень секретности, дату операции и другие служебные атрибуты.

Для ограничения доступа к строкам были включены политики Row Level Security. Они определяют, какие документы и операции доступны конкретному пользователю.

Листинг 5 – Включение RLS и создание политик доступа

```
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents FORCE ROW LEVEL SECURITY;

ALTER TABLE document_operations ENABLE ROW LEVEL SECURITY;
ALTER TABLE document_operations FORCE ROW LEVEL SECURITY;

CREATE POLICY documents_select_policy
ON documents
FOR SELECT
TO document_employee
USING
(
    secrecy_level IS NULL
    OR responsible_employee = current_user
);

CREATE POLICY documents_update_policy
ON documents
FOR UPDATE
TO document_employee
USING
(
    responsible_employee = current_user
)
WITH CHECK
(
    responsible_employee = current_user
);

CREATE POLICY operations_select_policy
ON document_operations
FOR SELECT
TO document_employee
USING
(
    employee_name = current_user
    OR EXISTS
    (
        SELECT 1
        FROM documents d
        WHERE d.document_number = document_operations.document_number
        AND d.responsible_employee = current_user
    )
);
```

```
CREATE POLICY operations_update_policy
ON document_operations
FOR UPDATE
TO document_employee
USING
(
    employee_name = current_user
)
WITH CHECK
(
    employee_name = current_user
);
```

Политика `documents_select_policy` разрешает чтение всех несекретных документов и секретных документов, где текущий пользователь является ответственным. Политика `documents_update_policy` разрешает изменение только документов, за которые отвечает текущий пользователь. Для таблицы `document_operations` чтение разрешается для собственных операций пользователя и операций по документам, за которые он назначен ответственным. Изменение операций разрешено только для собственных записей пользователя.

3.4 Проверка разграничения доступа

Проверка разграничения доступа выполнялась для пользователей `Сотрудник_В` и `Сотрудник_А`. Проверка проводилась путем подключения к базе данных от имени соответствующего пользователя и выполнения SQL-запросов к таблицам `documents` и `document_operations`.

Сначала была выполнена проверка доступа пользователя `Сотрудник_В` к таблице `documents`. Согласно заданной политике, данный пользователь должен видеть все несекретные документы, а также секретный документ `DOC-006`, за который он назначен ответственным. При этом чужие секретные документы не должны отображаться в результате запроса.

Листинг 6 – Проверка доступа пользователя `Сотрудник_В`

```
SELECT
    document_number,
    document_type,
    responsible_employee,
    internal_department,
    effective_from,
    effective_until,
    secrecy_level
FROM documents
ORDER BY document_number;
```

Результат выполнения запроса представлен на рисунке 2.

```
Lab1 > sql > report_check_B.sql
1 \encoding UTF8
2 \set pager off
3
4 \echo '=== Текущий пользователь ==='
5 SELECT current_user;
6
7 SELECT
8     document_number,
9     document_type,
10    responsible_employee,
11    internal_department,
12    effective_from,
13    effective_until,
14    secrecy_level
15 FROM documents
16 ORDER BY document_number;
17
```

```
root@GEORGUL-GAMER:/mnt/c/Users/Georgul/Documents/8_sem/SUBD/Lab1# PGPASSWORD=lab1pass psql -h 127.0.0.1 -p 15432 -U "Сотрудник_В"
-d subd_lab1 -P pager=off
-f sql/report_check_B.sql 2>&1 | tee logs/08_09_user_v_check.log
Pager usage is off.
=== Текущий пользователь ===
current_user
-----
Сотрудник_В
(1 row)

document_number | document_type | responsible_employee | internal_department | effective_from | effective_until | secrecy_level
-----
DOC-001         | Регламент     | Сотрудник_А        | Подразделение_1    | 2024-01-10     | 2026-01-10     |
DOC-003         | Инструкция    | Сотрудник_Б        | Подразделение_2    | 2024-03-01     | 2025-03-01     |
DOC-005         | Приказ        | Сотрудник_В        | Подразделение_3    | 2024-05-01     | 2025-05-01     |
DOC-006         | Служебная записка | Сотрудник_В        | Подразделение_3    | 2024-06-01     | 2026-06-01     | секретно
DOC-008         | Финансовая справка | Сотрудник_Д        | Подразделение_5    | 2024-08-01     | 2025-08-01     |
(5 rows)
```

Рисунок 2 – Документы, доступные пользователю Сотрудник_В

Из результата видно, что пользователь Сотрудник_В получил доступ к несекретным документам DOC-001, DOC-003, DOC-005 и DOC-008, а также к секретному документу DOC-006, где он является ответственным. Чужие секретные документы DOC-002, DOC-004 и DOC-007 в результате запроса отсутствуют.

Затем была проверена возможность изменения срока действия документа. Пользователь Сотрудник_В должен иметь возможность изменять поле effective_until только для документов, за которые он назначен ответственным.

Листинг 7 – Изменение срока действия документа пользователем Сотрудник В

```
UPDATE documents
SET effective_until = DATE '2026-10-01'
WHERE document_number = 'DOC-006';

SELECT document_number, responsible_employee, effective_until, secrecy_level
FROM documents
WHERE document_number = 'DOC-006';
```

Результат изменения срока действия собственного документа представлен на рисунке 3.

```
> sql > report_change_date_B.sql
\encoding UTF8
\set pager off

\echo '=== Текущий пользователь ==='
SELECT current_user;

UPDATE documents
SET effective_until = DATE '2026-10-01'
WHERE document_number = 'DOC-006';

SELECT document_number, responsible_employee, effective_until, secrecy_level
FROM documents
WHERE document_number = 'DOC-006';
```

```
root@GEORGUL-GAMER:/mnt/c/Users/Georgul/Documents/8_sem/SUBD/Lab1# PGPASSWORD=lab1pass psql -h 127.0.0.1 -p 15432 -U "Сотрудник_В" -d subd_lab1 -P pager=off -f sql/report_change_date_B.sql 2>&1 | tee logs/08_09_user_v_check.log
Pager usage is off.
=== Текущий пользователь ===
current_user
-----
Сотрудник_В
(1 row)

UPDATE 1
document_number | responsible_employee | effective_until | secrecy_level
-----
DOC-006         | Сотрудник_В        | 2026-10-01     | секретно
(1 row)
```

Рисунок 3 – Изменение срока действия документа DOC-006

Запрос был выполнен успешно, так как документ DOC-006 относится к пользователю Сотрудник_В. Таким образом, правило изменения срока действия документов для ответственного сотрудника выполняется.

Для проверки запрета на изменение недоступных атрибутов была выполнена попытка изменить степень секретности документа. Согласно матрице доступа, пользователь не должен иметь права изменять поле `secrecy_level`.

Листинг 8 – Запрет изменения степени секретности документа для пользователя Сотрудник В

```
UPDATE documents
SET secrecy_level = 'секретно'
WHERE document_number = 'DOC-006';
```

Результат выполнения запроса представлен на рисунке 4.

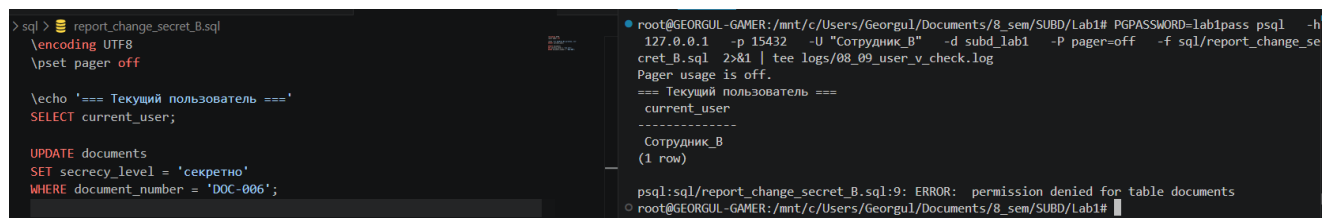


Рисунок 4 – Запрет изменения степени секретности документа

Запрос был отклонен, так как для роли `document_employee` не выдавалось право UPDATE на столбец `secrecy_level`. Это подтверждает корректность столбцового ограничения доступа.

После этого была проверена работа политик доступа для таблицы `document_operations`. Пользователь `Сотрудник_В` должен видеть операции, которые выполнил сам, а также операции по документам, за которые он назначен ответственным.

Листинг 9 – Операции, доступные для пользователя Сотрудник В

```
SELECT
employee_name,
document_number,
change_date,
operation_description
FROM document_operations
ORDER BY document_number, change_date;
```

Результат выполнения запроса представлен на рисунке 5.

```

> sql > report_get_ops_B.sql
\encoding UTF8
\pset pager off

\echo '=== Текущий пользователь ==='
SELECT current_user;

SELECT
employee_name,
document_number,
change_date,
operation_description
FROM document_operations
ORDER BY document_number, change_date;

```

```

root@GEORGUL-GAMER:/mnt/c/Users/Georgul/Documents/8_sem/SUBD/Lab1# PGPASSWORD=lab1pass psql -h 127.0.0.1 -p 15432
-U "Сотрудник_В" -d subd_lab1 -P pager=off -f sql/report_get_ops_B.sql 2>&1 | tee logs/08_09_user_v_check.log
Pager usage is off.
=== Текущий пользователь ===
current_user
-----
Сотрудник_В
(1 row)

employee_name | document_number | change_date | operation_description
-----
Сотрудник_В | DOC-002 | 2024-02-03 14:20:00 | Регистрация входящей операции, уточнено
Сотрудник_В | DOC-005 | 2024-05-02 08:45:00 | Создание приказа
Сотрудник_Д | DOC-006 | 2024-06-02 16:10:00 | Согласование документа
(3 rows)

```

Рисунок 5 – Операции, доступные пользователю Сотрудник_В

Из результата видно, что пользователь Сотрудник_В видит собственные операции по документам DOC-002 и DOC-005, а также операцию пользователя Сотрудник_Д по документу DOC-006. Появление операции пользователя Сотрудник_Д является корректным, так как документ DOC-006 закреплен за пользователем Сотрудник_В. Следовательно, политика доступа разрешает ответственному сотруднику просматривать операции по закрепленным за ним документам, но не дает права изменять чужие операции.

Также была проверена попытка изменения чужой операции по документу, за который пользователь является ответственным. Такое изменение должно быть запрещено, так как пользователь может изменять только собственные записи в отношении document_operations.

Листинг 10 – Запрет изменения чужой операции для пользователя Сотрудник_В

```

UPDATE document_operations
SET operation_description = 'Попытка изменить чужую операцию'
WHERE employee_name = 'Сотрудник_Д'
AND document_number = 'DOC-006';

```

Результат выполнения запроса представлен на рисунке 6.

```

1 > sql > report_change_other_B.sql
2 \encoding UTF8
3 \pset pager off
4
5 \echo '=== Текущий пользователь ==='
6 SELECT current_user;
7
8 UPDATE document_operations
9 SET operation_description = 'Попытка изменить чужую операцию'
10 WHERE employee_name = 'Сотрудник_Д'
11 AND document_number = 'DOC-006';

```

```

root@GEORGUL-GAMER:/mnt/c/Users/Georgul/Documents/8_sem/SUBD/Lab1# PGPASSWORD=lab1pass psql -h 127.0.0.1 -p 15432
-U "Сотрудник_В" -d subd_lab1 -P pager=off -f sql/report_change_other_B.sql 2>&1 | tee logs/08_09_user_v_check.log
Pager usage is off.
=== Текущий пользователь ===
current_user
-----
Сотрудник_В
(1 row)

UPDATE 0

```

Рисунок 6 – Запрет изменения чужой операции

Результат UPDATE 0 означает, что ни одна строка не была изменена. Политика RLS не разрешила пользователю изменить запись, где employee_name не совпадает с current_user.

Дополнительно была выполнена проверка для пользователя Сотрудник_А. Пользователь должен видеть все несекретные документы, собственный секретный документ DOC-002 и операции по документам, за которые он назначен ответственным.

Листинг 11 – Документы и операции, доступные пользователю Сотрудник_А

```
SELECT
    document_number,
    document_type,
    responsible_employee,
    secrecy_level
FROM documents
ORDER BY document_number;
SELECT
    employee_name,
    document_number,
    change_date,
    operation_description
FROM document_operations
ORDER BY document_number, change_date;
```

Результат проверки пользователя Сотрудник_А представлен на рисунке 7.

The screenshot shows a terminal window with the following content:

```
1 > sql > report_get_docs_A.sql
2 \encoding UTF8
3 \pset pager off
4 \echo '=== Текущий пользователь ==='
5 SELECT current_user;
6
7
8 SELECT
9     document_number,
10    document_type,
11    responsible_employee,
12    secrecy_level
13 FROM documents
14 ORDER BY document_number;
15 SELECT
16     employee_name,
17     document_number,
18     change_date,
19     operation_description
20 FROM document_operations
21 ORDER BY document_number, change_date;
```

The output shows the current user is 'Сотрудник_А'. The first table lists documents accessible to the user:

document_number	document_type	responsible_employee	secrecy_level
DOC-001	Регламент	Сотрудник_А	
DOC-002	План реагирования	Сотрудник_А	секретно
DOC-003	Инструкция	Сотрудник_Б	
DOC-005	Приказ	Сотрудник_В	
DOC-008	Финансовая справка	Сотрудник_Д	

The second table lists operations performed by the user:

employee_name	document_number	change_date	operation_description
Сотрудник_А	DOC-001	2024-01-11 10:00:00	Создание документа
Сотрудник_Б	DOC-001	2024-01-12 11:30:00	Проверка документа
Сотрудник_А	DOC-002	2024-02-02 09:15:00	Создание секретного документа
Сотрудник_В	DOC-002	2024-02-03 14:20:00	Регистрация входящей операции, уточнено

Рисунок 7 – Документы и операции, доступные пользователю Сотрудник_А

Проверка показала, что пользователь Сотрудник_А видит несекретные документы и секретный документ DOC-002, за который он назначен ответственным. В таблице операций пользователь видит собственные операции и операции по своим документам. Следовательно, политики безопасности строк работают согласно разработанной матрице доступа.

3.5 Итоговая матрица доступа

По результатам реализации и проверки была сформирована итоговая матрица доступа. Она совпадает с исходной матрицей для рассматриваемой роли сотрудника, так как все предусмотренные ограничения удалось реализовать средствами PostgreSQL.

Таблица 4 – Итоговая матрица доступа

Отношение	Атрибут или условие	Сотрудник
documents	document number	R
documents	document type	R
documents	responsible employee	R
documents	document content	R
documents	internal_department	R
documents	effective_from	R
documents	effective_until	R/U
documents	secrecy level	R
document_operations	employee_name	R
document_operations	document number	R
document_operations	change date	R
document_operations	operation description	R/U
documents	Видимые строки	Все несекретные документы; секретные документы только при <code>responsible_employee = current_user</code>
document_operations	Видимые строки	Свои операции; операции по документам, за которые пользователь ответственен
documents	Ограничение изменения	<code>effective_until</code> можно изменять только для документов, где <code>responsible_employee = current_user</code>
document_operations	Ограничение изменения	<code>operation_description</code> можно изменять только для записей, где <code>employee_name = current_user</code>

Для реализации вертикального ограничения доступа использовались столбцовые привилегии GRANT. Для реализации горизонтального ограничения доступа использовались политики Row Level Security. В результате пользователи могут читать только допустимые строки и изменять только разрешенные атрибуты.

При этом необходимо учитывать общее ограничение применяемого подхода: GRANT управляет доступом к объектам и столбцам, а RLS — доступом к строкам. Поэтому при более сложной схеме с несколькими ролями итоговые права пользователя могут расширяться за счет объединения привилегий и политик.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Какой тип разграничения доступа, с точки зрения грануляции, реализован в СУБД, на которой выполнялась работа?

В PostgreSQL поддерживается несколько уровней грануляции доступа. Во-первых, доступ может задаваться на уровне объектов базы данных: таблиц, схем, баз данных, функций, последовательностей и других объектов. Во-вторых, для таблиц возможно разграничение на уровне отдельных столбцов. В-третьих, PostgreSQL поддерживает разграничение доступа на уровне строк с помощью механизма Row Level Security.

В выполненной работе использовалась комбинация столбцового и строкового разграничения доступа. Столбцовые ограничения задавались с помощью GRANT, а ограничения на уровне строк — с помощью политик RLS.

4.2 Какой тип гранулированного доступа реализует предложение GRANT?

Предложение GRANT реализует разграничение доступа на уровне объектов базы данных и, для некоторых операций, на уровне отдельных столбцов таблицы. С помощью GRANT можно выдать права на таблицу целиком, например SELECT или UPDATE, либо выдать право только на конкретный столбец.

Например, в работе право изменения даты окончания действия документа задавалось не для всей таблицы documents, а только для столбца effective_until. Это позволило запретить пользователям изменение остальных атрибутов документа.

4.3 Какой тип гранулированного доступа реализует предложение POLICY?

Предложение POLICY используется для реализации разграничения доступа на уровне строк таблицы. В PostgreSQL политики применяются в рамках механизма Row Level Security.

Политика содержит условие, которое проверяется для каждой строки. Если условие выполняется, строка доступна пользователю для соответствующей

операции. Если условие не выполняется, строка скрывается от пользователя или не может быть изменена.

4.4 Что происходит при совмещении нескольких политик доступа в отношении одной ячейки (записи) для одного и того же пользователя?

Если для одной таблицы и одного пользователя применимо несколько разрешающих политик, PostgreSQL объединяет их по принципу логического «ИЛИ». Это означает, что доступ к строке будет разрешен, если выполняется хотя бы одна из применимых политик.

Такое поведение необходимо учитывать при проектировании разграничения доступа. При наличии нескольких ролей или нескольких политик итоговый доступ пользователя может оказаться шире, чем ожидалось при рассмотрении каждой политики отдельно.

4.5 Кто обладает правом разграничения прав доступа в рассматриваемой СУБД? Какие права для этого необходимы?

В PostgreSQL правами доступа управляет владелец объекта или суперпользователь. Владелец таблицы может выдавать и отзывать права на нее с помощью GRANT и REVOKE. Суперпользователь может управлять правами независимо от владельца объекта.

Для создания, изменения и удаления политик RLS также необходимы права владельца таблицы или права суперпользователя. Включение и отключение Row Level Security выполняется командой ALTER TABLE, поэтому также требует прав владельца соответствующей таблицы.

4.6 Есть ли какие-либо ограничения применения политик?

Да, политики RLS имеют ряд ограничений. Во-первых, они могут усложнять администрирование базы данных, так как итоговый доступ пользователя зависит не только от выданных привилегий, но и от условий политик. Во-вторых,

политики могут влиять на производительность, поскольку условия доступа проверяются при выполнении запросов.

Также важно учитывать, что суперпользователи PostgreSQL могут обходить ограничения RLS. Кроме того, владелец таблицы по умолчанию также может обходить политики, если для таблицы не задан режим FORCE ROW LEVEL SECURITY.

4.7 К каким объектам может быть применено предложение GRANT?

GRANT может применяться к различным объектам PostgreSQL: базам данных, схемам, таблицам, представлениям, последовательностям, функциям, процедурам, языкам, табличным пространствам и ролям.

В рамках данной работы GRANT применялся к таблицам и отдельным столбцам таблиц. Также оператор GRANT использовался для включения пользователей в групповую роль `document_employee`.

4.8 К каким объектам может быть применено предложение POLICY?

Политики RLS применяются к таблицам. В PostgreSQL политика создается для конкретной таблицы и определяет, какие строки этой таблицы доступны пользователю при выполнении операций SELECT, INSERT, UPDATE или DELETE.

В выполненной работе политики были применены к таблицам `documents` и `document_operations`.

4.9 Какие пользователи и роли изначально определены в рассматриваемом сервере СУБД? У каких из них заданы пароли по умолчанию?

В PostgreSQL набор начальных ролей зависит от способа установки и настройки сервера. Обычно при инициализации создается административная роль, соответствующая владельцу кластера баз данных. В используемом контейнере

PostgreSQL была создана административная учетная запись `lab_admin`, параметры которой задавались при запуске контейнера через переменные окружения.

Служебные роли PostgreSQL, такие как `pg_read_all_data`, `pg_write_all_data`, `pg_monitor` и другие, используются для предоставления специальных наборов прав. Как правило, такие роли не являются учетными записями для входа и не имеют паролей по умолчанию.

4.10 Как разграничить доступ пользователей к функциям? Какие права при этом должны быть также предоставлены пользователю и должны ли?

Доступ к функциям в PostgreSQL можно разграничить с помощью права `EXECUTE`. Для этого используется оператор `GRANT EXECUTE ON FUNCTION`. Если функция находится в определенной схеме, пользователю также может потребоваться право `USAGE` на эту схему.

Пример:

```
GRANT USAGE ON SCHEMA public TO document_employee;
```

```
GRANT EXECUTE ON FUNCTION function_name() TO document_employee;
```

Если пользователь не имеет права `EXECUTE` на функцию, он не сможет вызвать ее напрямую. При этом необходимо учитывать права, с которыми функция выполняется: `SECURITY INVOKER` или `SECURITY DEFINER`.

4.11 Можно ли дать пользователям доступ на чтение к отдельным таблицам системного каталога сервера СУБД?

Да, доступ на чтение к отдельным объектам системного каталога может быть предоставлен, если это необходимо для выполнения задач пользователя. В PostgreSQL многие представления системного каталога доступны для чтения обычным пользователям, но часть сведений может быть ограничена.

Для расширения доступа можно использовать специальные системные роли, например роли семейства `pg_read_all_*`. Однако выдавать такие права следует осторожно, так как системный каталог может содержать сведения о

структуре базы данных, ролях, правах и других объектах, которые могут быть чувствительными с точки зрения безопасности.

5 ВЫВОДЫ

В ходе выполнения лабораторной работы была разработана схема базы данных для варианта № 8, включающая таблицы `documents` и `document_operations`. Для таблиц были заданы первичные ключи, альтернативный ключ, внешний ключ и ограничения целостности.

Была разработана матрица доступа, описывающая права сотрудника на чтение и изменение атрибутов, а также ограничения на видимость строк. Для реализации матрицы использовались встроенные средства PostgreSQL: роли, привилегии GRANT/REVOKE и политики безопасности строк Row Level Security.

В результате были созданы обезличенные пользователи `Сотрудник_А`, `Сотрудник_Б`, `Сотрудник_В`, `Сотрудник_Г` и `Сотрудник_Д`, объединенные общей ролью `document_employee`. С помощью столбцовых привилегий было ограничено изменение отдельных атрибутов: `effective_until` в таблице `documents` и `operation_description` в таблице `document_operations`.

Проверка показала, что сотрудник видит все несекретные документы и только те секретные документы, за которые он назначен ответственным. Также сотрудник видит собственные операции и операции по документам, за которые он отвечает, но не может изменять чужие операции. Таким образом, реализованное разграничение доступа соответствует условиям варианта.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Моргунов, Е. П. PostgreSQL. Основы языка SQL : учебное пособие / Е. П. Моргунов ; под ред. Е. В. Рогова, П. В. Лузанова. — Санкт-Петербург : БХВ-Петербург, 2018. — 336 с.
2. Полтавцева, М. А. Практикум по построению защищенных баз данных: учебное пособие / М. А. Полтавцева [и др.]. — Санкт-Петербург : ПОЛИТЕХ-ПРЕСС, 2023. — 182 с.
3. Рогов, Е. В. PostgreSQL 15 изнутри / Е. В. Рогов. — Москва : ДМК Пресс, 2023. — 662 с.
4. The PostgreSQL Global Development Group. PostgreSQL Documentation: GRANT. — Режим доступа: <https://www.postgresql.org/docs/current/sql-grant.html>
5. The PostgreSQL Global Development Group. PostgreSQL Documentation: Row Security Policies. — Режим доступа: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>